

8-Puzzle Search Algorithms in Artificial Intelligence

1. Problem Description

- 3x3 sliding puzzle with 8 tiles and one empty space.
- Goal: Arrange tiles in order (1, 2, 3, 4, 5, 6, 7, 8, 0).

2. State & Successors

- **State:** tuple of 9 integers.
- **Successors:** `get_successors(state)` generates valid moves (`new_state`, `action`)

3. Algorithms Implemented

- BFS, DFS, UCS, IDS, A*, Hill Climbing
- **Heuristic:** Manhattan distance for A* & Hill Climbing.

4. Setup and Comparison

- Initial state: (1, 2, 3, 4, 0, 6, 7, 5, 8)
- Metrics: Steps, Time(ms), Nodes Expanded, Space Used, Path Cost, Optimal

5. Results

Algorithm	Steps	Time (ms)	Nodes Expanded	Space Used	Path Cost	Optimal
BFS	2	0.0	9	9	2	Yes
DFS	2	471.15	140143	140143	2	Yes
UCS	2	0.0	14	14	2	Yes
IDS	2	0.0	13	13	2	Yes
A*	2	0.0	3	3	2	Yes
Hill Climbing	2	0.0	3	3	2	Yes

6. Conclusion

- **Best Algorithm:** A* for efficiency and optimal solution.

- BFS & UCS reliable but use more memory.
- Hill Climbing fast but may fail on complex states, while DFS is slow.

Team Contributions

- 1- Mariam Mahmoud Ibrahim Elzawawy , 2023211 , puzzle.py and main.oy
- 2- Rana Zaghlool Ahmed Dorra , 2023083 , Searching Algorithms
- 3- Menna Gamal Abd-Ellateef Salama ,2023266 , README and report